CogLean

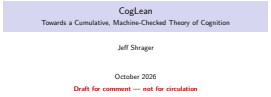
Towards a Cumulative, Machine-Checked Theory of Cognition

Jeff Shrager

October 2026

Draft for comment — not for circulation

CogLean

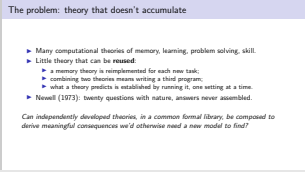


This talk goes with the paper of the same name, which is a draft for comment. It is not for circulation, and it hasn't been carefully checked yet. The paper introduces CogLean, a library of cognitive theory in the Lean proof assistant, and argues for a way of doing theoretical cognitive science: theories stated once, their consequences proved, and independently developed theories composed so that new results follow from old ones. The central example is multi-digit addition, but I want to be clear from the start that it is a demonstration, not a new theory of arithmetic.

The problem: theory that doesn't accumulate

- Many computational theories of memory, learning, problem solving, skill.
- Little theory that can be **reused**:
  - a memory theory is reimplemented for each new task;
  - combining two theories means writing a third program;
  - what a theory predicts is established by running it, one setting at a time.
- Newell (1973): twenty questions with nature, answers never assembled.

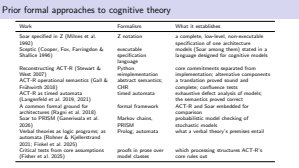
Can independently developed theories, in a common formal library, be composed to derive meaningful consequences we'd otherwise need a new model to find?



The problem is an old one. Cognitive science has many computational theories, but little theory that can be reused. A theory of memory developed for verbal learning has to be reimplemented before it can take part in a model of arithmetic. Combining two theories means writing a third program and exploring it by simulation. And what a theory predicts is established by running it, at one setting at a time. Newell complained in 1973 that we play twenty questions with nature and never assemble the answers. The question for this talk is whether a common formal library can change that: whether theories in such a library can be composed to give meaningful consequences that we would otherwise need a whole new model to find.

Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of one architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Frühwirth 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar to PRISM (Ganeriwala et al. 2026)	Markov chains, PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rohner & Kjellerstrand 2021; Finkel et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2025)	proofs in prose over model classes	which processing structures ACT-R's core rules out

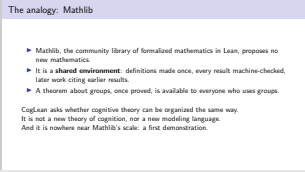


CogLean is far from the first formal treatment of cognitive theory, and it's worth seeing what came before. In 1992 Milnes and colleagues specified Soar in the Z notation: complete, but low-level and not executable. Cooper, Fox, Farringdon and Shallice argued in 1996 for languages designed for specifying cognitive models, and built one, Sceptic. Stewart and West rebuilt ACT-R to separate its core commitments from its implementation. For ACT-R there's a formal operational semantics with a translation proved sound and complete, and a line of work translating ACT-R models into timed automata for exhaustive defect checking. Ragni and colleagues proposed a common formal ground into which ACT-R and Soar can be embedded. Most recently, Soar models have been compiled for the PRISM probabilistic model checker. Outside architectures, verbal theories have been formalized as logic programs and as automata, and Fisher and colleagues derived critical tests of ACT-R from its core assumptions. Almost all of this work formalizes one architecture or one model at a time, and establishes properties of that system. That's the contrast I'll come back to.

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.



Mathematics faced a version of this problem and has made real progress on it. Mathlib, the community library for the Lean proof assistant, doesn't propose new mathematics. It's a shared environment: definitions are made once, every result is checked by machine, and later work builds on earlier results by citing them. A theorem about groups, once proved, is available to everyone who works with groups. CogLean asks whether cognitive theory can be organized the same way. It is not a new theory of cognition, and it's not a new modelling language either. And it's nowhere near Mathlib's scale; this is a first demonstration that the approach can work.

CogLean's layers

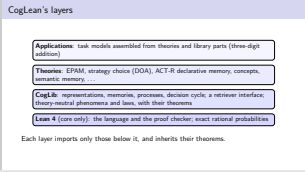
Applications: task models assembled from theories and library parts (three-digit addition)

Theories: EPAM, strategy choice (DOA), ACT-R declarative memory, concepts, semantic memory, ...

CogLib: representations, memories, processes, decision cycle; a retriever interface; theory-neutral phenomena and laws, with their theorems

Lean 4 (core only): the language and the proof checker; exact rational probabilities

Each layer imports only those below it, and inherits their theorems.



CogLean has four layers. At the bottom is Lean, core only, with exact rational probabilities. Above it is CogLib, the general library: representations such as feature codes and distributions; memory structures such as discrimination nets and an activation memory; processes such as sampled retrieval and a decision cycle with impasses; an interface that any memory can implement; and theory-neutral statements of phenomena and laws, with theorems about them. Above that are the theories, each stated as choices of library mechanisms. And at the top are applications, models of particular tasks assembled from theories and library parts. Each layer imports only those below it, and inherits their theorems.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("x is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Four kinds of object, kept apart

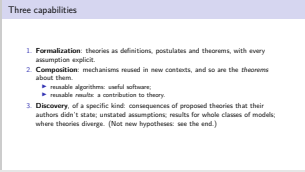
1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("r is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Four kinds of object have to be kept apart. Mechanisms and their postulates are definitions. Phenomena are predicates, stated without reference to any mechanism; a predicate asserts nothing, it's a target. Theorems say that a mechanism produces behaviour satisfying a predicate. And conditions are the hypotheses under which a theorem holds: some situational, some empirical premises we haven't explained yet, some just well-formedness. One rule follows: a theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction of any model. The library's catalog flags those mechanically.

Three capabilities

1. **Formalization**: theories as definitions, postulates and theorems, with every assumption explicit.
2. **Composition**: mechanisms reused in new contexts, and so are the *theorems* about them.
   - reusable algorithms: useful software;
   - reusable *results*: a contribution to theory.
3. **Discovery**, of a specific kind: consequences of proposed theories that their authors didn't state; unstated assumptions; results for whole classes of models; where theories diverge. (Not new hypotheses: see the end.)



The argument has three parts, in increasing order of importance. Formalization: theories stated precisely, with every assumption explicit. Composition: mechanisms reused in new contexts, and, crucially, the theorems about them reused too. A library of reusable cognitive algorithms would just be useful software; a library of reusable results is a contribution to theory. And discovery, which matters most: consequences the original authors didn't state, assumptions they didn't know they were making, results that hold for whole classes of models, and conditions under which theories diverge.

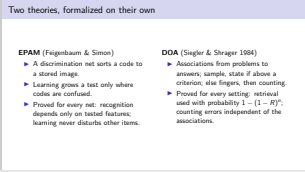
Two theories, formalized on their own

EPAM (Feigenbaum & Simon)

- A discrimination net sorts a code to a stored image.
- Learning grows a test only where codes are confused.
- Proved for every net: recognition depends only on tested features; learning never disturbs other items.

DOA (Siegler & Shrager 1984)

- Associations from problems to answers; sample, state if above a criterion; else fingers, then counting.
- Proved for every setting: retrieval used with probability $1 - (1 - R)^n$; counting errors independent of the associations.



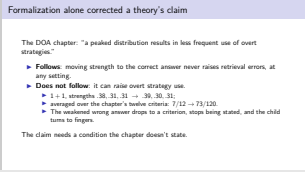
The demonstration uses two theories, each formalized on its own first. EPAM, Feigenbaum and Simon's model, recognizes a stimulus by sorting its code down a discrimination net; learning grows a test only where two codes are confused. Proved for every net: recognition depends only on the features tested, and learning never disturbs what's already learned. And the distributions-of-associations model that Bob Siegler and I proposed in 1984: a child samples answers in proportion to strength, states one that clears a confidence criterion, and otherwise falls back on fingers and counting. Proved for every setting: retrieval is used with probability one minus one minus R to the n, and counting errors don't depend on the associations at all, which was the chapter's nonintuitive prediction.

Formalization alone corrected a theory's claim

The DOA chapter: "a peaked distribution results in less frequent use of overt strategies."

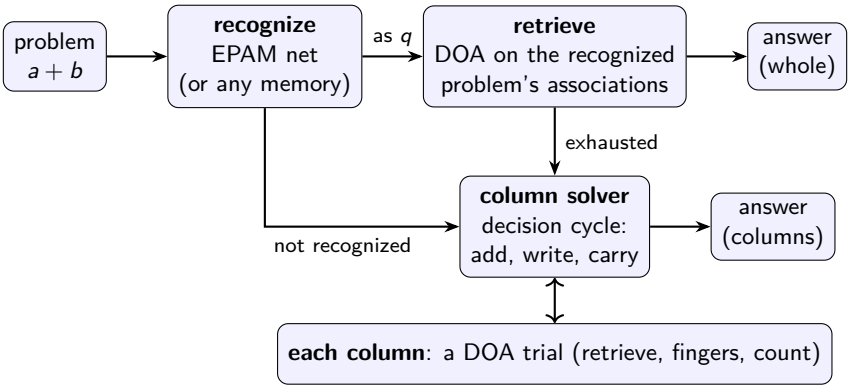
- **Follows**: moving strength to the correct answer never raises retrieval errors, at any setting.
- **Does not follow**: it can *raise* overt strategy use.
  - $1 + 1$, strengths $.38, .31, .31 \rightarrow .39, .30, .31$;
  - averaged over the chapter's twelve criteria: $7/12 \rightarrow 73/120$.
  - The weakened wrong answer drops to a criterion, stops being stated, and the child turns to fingers.

The claim needs a condition the chapter doesn't state.



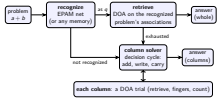
Before any composition, formalization alone corrected one of the theories, and it's one of my own. The chapter says that a peaked distribution results in less frequent use of overt strategies. Half of that follows from the model: moving strength to the correct answer never raises retrieval errors, at any setting. Half doesn't. Peaking can raise overt strategy use. Take one plus one with strengths point three eight, point three one, point three one, and move one hundredth from a wrong answer to the correct one. Averaged over the chapter's own twelve criteria, overt strategy use goes up, from seven twelfths to seventy-three over one twenty. The weakened wrong answer drops to one of the criteria, stops being stated, and the child turns to fingers. So the claim needs a condition the chapter never stated.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.

The application: three-digit addition



A demonstration, not a theory of arithmetic. In Lean, the composition is three definitions.

Here's the application. A problem is recognized by an EPAM net, or by any memory. If it's recognized as a familiar problem q, the answer is retrieved from q's distribution of associations. If retrieval is exhausted, or nothing is recognized, a column solver takes over. It runs on the library's decision cycle, adding, writing and carrying, and each column's addition is itself a full single-digit trial from the strategy-choice model. In Lean the whole composition is three definitions. Again: arithmetic is the demonstration. It's a familiar task that needs several mechanisms, and two of the theories already existed.

No answer supplied

- The earlier worked case used a backup that returned the correct sum: `Dist.pure p.sum`.
  - Enough to instantiate theorems about *every* backup;
  - but it supplies the answer instead of computing it.
- In the integrated model, **nothing is handed the answer**.
  - 250 for $150 + 100$ is assembled by the solver from three DOA trials and a carry rule, and is never certain.
  - Correct knowledge enters only as single-digit associations and the carry rule.

So composition has to work twice: the mechanisms run together, and their theorems transfer.

No answer supplied

- The earlier worked case used a backup that returned the correct sum: `Dist.pure p.sum`.
  - Enough to instantiate theorems about *every* backup;
  - but it supplies the answer instead of computing it.
- In the integrated model, **nothing is handed the answer**.
  - 250 for 150 + 100 is assembled by the solver from three DOA trials and a carry rule, and is never certain.
  - Correct knowledge enters only as the single-digit associations and the carry rule.

So composition has to work twice: the mechanisms run together, and their theorems transfer.

One detail matters a lot for whether this counts as evidence. An earlier version of the worked case used a backup that simply returned the correct sum. That's enough to instantiate theorems that hold for every backup, but it supplies the answer instead of computing it. In the integrated model nothing is handed the answer. Two hundred fifty for one fifty plus one hundred is assembled by the solver from three single-digit trials and a carry rule, and it's never certain. Correct knowledge enters only as the single-digit associations and the carry rule. So the composition has to work twice: the mechanisms have to run together, and their theorems have to transfer to the whole.

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's *distribution* of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer.

What actually happens when retrieval fails

1. **Recognize**: the EPAM net sorts the problem's code to a stored problem, possibly the wrong one.
2. **Retrieve**: DOA searches that problem's associations for an answer above the criterion.
3. **Fail**: if the search is exhausted, the backup is invoked.
4. **Solve**: the backup runs column addition on CogLib's general problem-solving machinery.
5. **Each column**: units, then tens, then hundreds; each digit sum is a separate DOA trial (retrieve, else fingers, else count).
6. **Combine**: each trial's *distribution* of answers feeds the rest of the calculation, with carrying and place value.
7. **Result**: a probability distribution over complete multi-digit answers, errors included.

`Dist.bind` composes these steps: it carries *every* possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer.

So what actually happens when whole-problem retrieval fails? Step by step. The EPAM net sorts the problem's code to a stored problem, possibly the wrong one. DOA searches that problem's associations for an answer above the criterion. If the search is exhausted, the backup is invoked. The backup runs column addition on the library's general problem-solving machinery: units, then tens, then hundreds, and each column's digit sum is a separate single-digit DOA trial, with retrieval, fingers or counting. Each trial produces a distribution of possible answers, and that whole distribution feeds the rest of the calculation, with carrying and place value. The result is a probability distribution over complete multi-digit answers, errors included. The operation that does the composing is bind: it carries every possible outcome, with its probability, through the steps that follow. Nothing is sampled, and nothing looks up the right answer.

The general machinery, and its arithmetic instance

CogLib (`Process/ProblemSolving`), task-general:

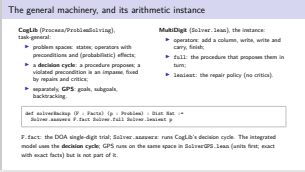
- problem spaces: states; operators with preconditions and (probabilistic) effects;
- a **decision cycle**: a procedure proposes; a violated precondition is an *impasse*, fixed by repairs and critics;
- separately, **GPS**: goals, subgoals, backtracking.

MultiDigit (`Solver.lean`), the instance:

- operators: add a column, write, write and carry, finish;
- `full`: the procedure that proposes them in turn;
- `lenient`: the repair policy (no critics).

```
def solverBackup (F : Facts) (p : Problem) : Dist Nat :=
  Solver.answers F.fact Solver.full Solver.lenient p
```

`F.fact`: the DOA single-digit trial; `Solver.answers`: runs CogLib's decision cycle. The integrated model uses the **decision cycle**; GPS runs on the same space in `SolverGPS.lean` (units first; exact with exact facts) but is not part of it.



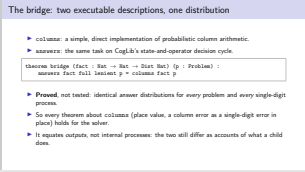
Here's the machinery underneath. The library's problem-solving module is task-general. It has a problem space, with states and operators that have preconditions and effects, and an effect can be a probability distribution. It has a decision cycle: a procedure proposes an operator; if its precondition is violated, that's an impasse, and repair heuristics and critics fix it. And, separately, it has GPS-style means-ends analysis, with goals, subgoals and backtracking. Multi-digit addition is one instance: operators to add a column, write, write and carry, and finish; a procedure, full, that proposes them in turn; and a repair policy, lenient. The backup is one call: answers, which runs the library's decision cycle, given the single-digit DOA trial, the procedure and the repair policy. One precision: the integrated model uses the decision cycle. GPS runs on the same problem space in a separate file, where it's proved to work units first and to be exact with exact facts, but it isn't part of the integrated model.

The bridge: two executable descriptions, one distribution

- `columns`: a simple, direct implementation of probabilistic column arithmetic.
- `answers`: the same task on CogLib's state-and-operator decision cycle.

```
theorem bridge (fact : Nat → Nat → Dist Nat) (p : Problem) :
    answers fact full lenient p = columns fact p
```

- **Proved**, not tested: identical answer distributions for *every* problem and *every* single-digit process.
- So every theorem about `columns` (place value, a column error as a single-digit error in place) holds for the solver.
- It equates *outputs*, not internal processes: the two still differ as accounts of what a child does.

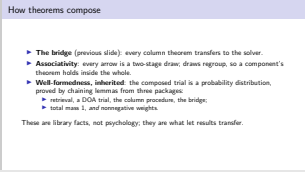


Now the bridge. There are two executable descriptions of column addition. One, columns, is a simple direct implementation of probabilistic column arithmetic. The other, answers, does the same task on the library's general state-and-operator decision cycle. The bridge theorem says they produce identical answer distributions, for every problem and for every single-digit process you plug in. That's proved, not established by testing examples. Its payoff is that every theorem proved about the simple version, like place value, or a column error being a single-digit error moved to its place, holds for the solver too. But be precise about what it equates: outputs, not internal processes. As accounts of what a child does, the two are still different. This is the library idea in miniature: results transfer between independently defined mechanisms.

How theorems compose

- **The bridge** (previous slide): every column theorem transfers to the solver.
- **Associativity**: every arrow is a two-stage draw; draws regroup, so a component's theorem holds inside the whole.
- **Well-formedness, inherited**: the composed trial is a probability distribution, proved by chaining lemmas from three packages:
  - retrieval, a DOA trial, the column procedure, the bridge;
  - total mass 1, *and* nonnegative weights.

These are library facts, not psychology; they are what let results transfer.



Three kinds of reuse make the composition deductive rather than just executable. A bridge theorem says the solver on the decision cycle answers exactly as a fixed column procedure does, for every problem and every single-digit process, so everything proved about the fixed procedure transfers. Associativity: every arrow is a two-stage draw of distributions, and draws regroup, so a theorem about a component holds where it sits inside the whole. And well-formedness is inherited: that the composed trial is a genuine probability distribution, total mass one and no negative weights, is proved by chaining lemmas from three packages. None of this is psychology. It's what lets the psychology transfer.

Confident errors

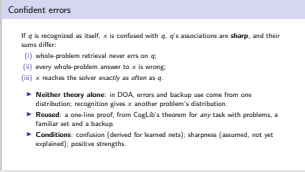
If q is recognized as itself, x is confused with q, q's associations are **sharp**, and their sums differ:

(i) whole-problem retrieval never errs on q;

(ii) every whole-problem answer to x is wrong;

(iii) x reaches the solver *exactly as often* as q.

- **Neither theory alone**: in DOA, errors and backup use come from one distribution; recognition gives x another problem's distribution.
- **Reused**: a one-line proof, from CogLib's theorem for *any* task with problems, a familiar set and a backup.
- **Conditions**: confusion (derived for learned nets); sharpness (assumed, not yet explained); positive strengths.

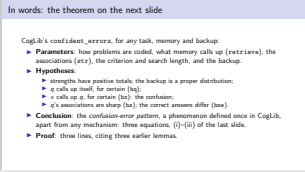


The first result is confident errors. Suppose a familiar problem q is recognized as itself, a stranger x is confused with it, q's associations are sharp, meaning only q's correct sum clears the criterion, and the two sums differ. Then retrieval never errs on q, every whole-problem answer to x is wrong, and x goes to the solver exactly as often as q does. Neither theory says this alone: in strategy choice, a problem's errors and its backup use come from the same distribution, so they go together. Recognition breaks the tie by handing x another problem's distribution. The proof is one line, applying a library theorem that holds for any task with problems, a familiar set and a backup. And the conditions are explicit: confusion, which is derived for learned nets; sharpness, which is assumed and not yet explained; and positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.



The next slide shows the actual Lean theorem, so here it is in words first. It's the library's version, which holds for any task, any memory and any backup. Its parameters are how problems are coded, what memory calls up for a code, the associations, the criterion and search length, and the backup. Its hypotheses: well-formedness of the strengths and the backup; the familiar problem calls up itself for certain; the stranger calls up the familiar problem for certain, which is the confusion; the familiar problem's associations are sharp; and the correct answers differ. The conclusion is the confusion-error pattern, which the library defines once, as a phenomenon, with no mechanism in it: the three equations from the previous slide. And the proof is three lines, each citing an earlier lemma.

The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). The multi-digit version is a one-line instance.

The theorem, in Lean

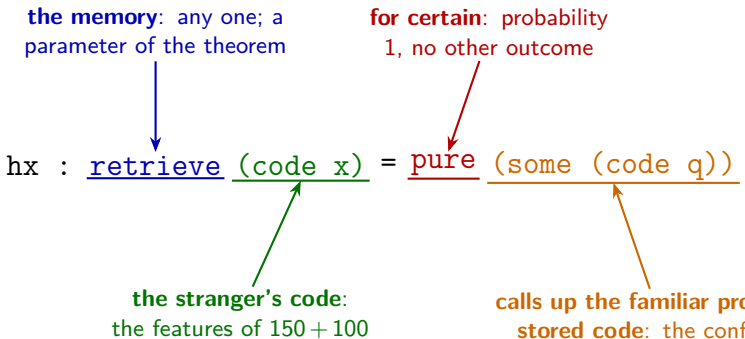
```
theorem confident_errors (correct : P → α) (q : P) (hT : ∀ p, total as (otr p) ≠ 0)
    (hbs : mass (backup s) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp otr cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve otr as cc s backup)
      (wholeAnswer correct) (wholeAnswer q) (correct q) :=
  refine ⟨confusion_given_neighbours_answer code retrieve otr as cc s backup
      (correct q) (hT _) hq he,
    wrong_eq_whole_of_sharp code retrieve otr as cc s backup
      (correct q) (hT _) hq he hs, ?_⟩
  rw [backup_prob_certain code retrieve otr as cc s backup q _ hq hbq hT,
    backup_prob_certain code retrieve otr as cc s backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist Q) (wholeAnswer : P → α → Real)
    (isWhole isBackup : Q → Real) (q : P) : Prop :=
  prob (trial q) (wholeAnswer q) = 0 ∧
    prob (trial q) (wholeAnswer q) = prob (trial q) isWhole ∧
    prob (trial q) isBackup = prob (trial q) isBackup
```

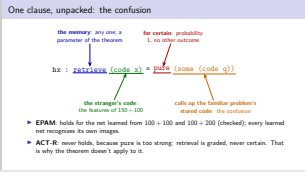
Verbatim from CogLib (Process/RecognizeRetrieve, Phenomena/ConfusionErrors). The multi-digit version is a one-line instance.

Here's the theorem itself, verbatim from the library. The first five lines are the hypotheses, each named: h T, the two backup hypotheses, h q, h x, h s and h n e. Then the conclusion: the trial has the confusion-error pattern. Then the proof. At the bottom is that pattern's definition, three equations joined by and, about observable outcomes only. The first clause is the lemma that a problem recognized as a sharp q gets only q's answer; the second is the lemma that all its whole-problem answers are then wrong; and the third rewrites both backup probabilities with the lemma that backup use after certain recognition depends only on what's recognized. Nothing in the statement mentions arithmetic, EPAM or ACT-R. The multi-digit model's theorem is a one-line instance of this one.

One clause, unpacked: the confusion



- **EPAM**: holds for the net learned from $100 + 100$ and $100 + 200$ (checked); every learned net recognizes its own images.
- **ACT-R**: *never* holds, because `pure` is too strong: retrieval is graded, never certain. That is why the theorem doesn't apply to it.



Let's unpack one clause, the confusion, h x. Read it left to right. Retrieve is the memory, and it's a parameter: the theorem holds for any memory you plug in. Code x is the stranger's feature code, here the features of one fifty plus one hundred. Pure means for certain: the distribution with a single outcome, at probability one. And some code q says the outcome is the familiar problem's stored code. So the clause says: this memory calls up the familiar problem, for certain, when given the stranger. For EPAM it holds: the net learned from one hundred plus one hundred and one hundred plus two hundred is checked to do exactly this. For ACT-R it never holds, and the culprit is the word pure: ACT-R's retrieval is graded and never certain. That single word is why the theorem applies to one memory and not the other.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

Equal is not low

- The theorem gives *equal* solver use for x and q, not *low* solver use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the confused problem reaches the solver with probability at most $(1 - r)^n$.
- With flat associations on q, the same theorem holds, and the errors are not confident.

Here $R_q = 45/46$, $n = 1$: the solver is reached $1/46$ of the time.

One precision matters here. The theorem says the confused problem reaches the solver exactly as often as the familiar one. It doesn't say that's rare. The rate is one minus R q to the n, where R q is the chance that one search of q's associations clears the criterion; and if R q is at least r, solver use is at most one minus r to the n. Both are proved. So whether these errors are confident, fluent and unhesitating, is a separate, quantitative matter. With flat associations the same theorem holds, but the errors wouldn't be confident. In our setting R q is forty-five over forty-six, so the solver is reached once in forty-six.

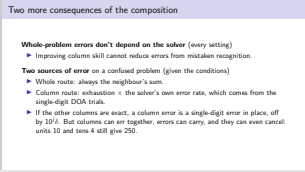
Two more consequences of the composition

Whole-problem errors don't depend on the solver (every setting)

- Improving column skill cannot reduce errors from mistaken recognition.

Two sources of error on a confused problem (given the conditions)

- Whole route: always the neighbour's sum.
- Column route: exhaustion $\times$ the solver's own error rate, which comes from the single-digit DOA trials.
- If the other columns are exact, a column error is a single-digit error in place, off by $10^j\delta$. But columns can err together, errors can carry, and they can even cancel: units 10 and tens 4 still give 250.

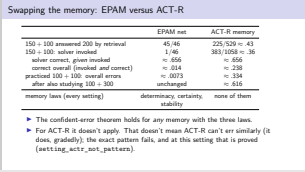


Two more consequences. First, whole-problem errors don't depend on the solver at all, at any setting. Psychologically that's a prediction about intervention: teaching column skill can't reduce errors that come from mistaken recognition; only learning that changes recognition or associations can. Second, a confused problem has two sources of error. The whole route always gives the neighbour's sum. The column route errs with probability equal to exhaustion times the solver's own error rate, which comes entirely from the single-digit trials. When only one column errs, the error is that single-digit error moved to its place. But that's a special case: columns can err together, errors propagate through carries, and they can even cancel. A units result of ten and a tens result of four still give two hundred fifty.

Swapping the memory: EPAM versus ACT-R

	EPAM net	ACT-R memory
$150 + 100$ answered 200 by retrieval	$45/46$	$225/529 \approx .43$
$150 + 100$: solver invoked	$1/46$	$383/1058 \approx .36$
solver correct, *given* invoked	$\approx .656$	$\approx .656$
correct overall (invoked *and* correct)	$\approx .014$	$\approx .238$
practiced $100 + 100$: overall errors	$\approx .0073$	$\approx .334$
after also studying $100 + 300$	unchanged	$\approx .616$
memory laws (every setting)	determinacy, certainty, stability	none of them

- The confident-error theorem holds for *any* memory with the three laws.
- For ACT-R it doesn't apply. That doesn't mean ACT-R can't err similarly (it does, gradedly); the exact pattern fails, and at this setting that is proved (`setting_actr_not_pattern`).

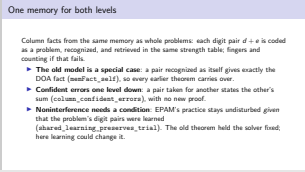


Because recognition is stated for any memory, we can swap EPAM's net for ACT-R's declarative memory and change nothing else. The confident-error pattern is proved for any memory with three laws: determinacy, certainty and stability. EPAM's net has all three at every setting; ACT-R's memory has none of them, also proved. So the theorem applies to EPAM and never to ACT-R. But be careful about what that means. It doesn't mean ACT-R can't make similar errors; at this setting it answers two hundred about forty-three percent of the time. What fails is the exact pattern: ACT-R's confusion is graded, and its solver use differs from the familiar problem's. A note on the solver rows: the solver is right about sixty-six percent of the time when it's invoked; the overall figure is that times how often it's invoked. It also predicts interference: studying a new problem raises the errors on a practiced one, which EPAM never does. All the numbers are exact, at one illustrative setting.

One memory for both levels

Column facts from the *same* memory as whole problems: each digit pair $d + e$ is coded as a problem, recognized, and retrieved in the same strength table; fingers and counting if that fails.

- **The old model is a special case**: a pair recognized as itself gives exactly the DOA fact (`memFact_self`), so every earlier theorem carries over.
- **Confident errors one level down**: a pair taken for another states the other's sum (`column_confident_errors`), with no new proof.
- **Noninterference needs a condition**: EPAM's practice stays undisturbed *given* that the problem's digit pairs were learned (`shared_learning_preserves_trial`). The old theorem held the solver fixed; here learning could change it.



So far the solver's single-digit facts came from their own table, separate from the memory that recognizes whole problems. A child has one memory, so we removed the separation. Each digit pair, say five plus zero, is coded as a problem in the same code space, recognized by the same memory, and retrieved from the same strength table, with fingers and counting if retrieval fails. Three things follow. First, if a pair is recognized as itself, its fact is exactly the old DOA fact, so the old model is a special case and every earlier theorem carries over. Second, a pair taken for another pair gives confident errors one level down, by the same general theorem, with no new proof. Third, a caution: the old noninterference theorem held the solver fixed while the net learned. With one memory, learning could change the solver too, so noninterference needs a condition: it holds when the problem's digit pairs were themselves learned, and that we prove.

One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	120/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct ($.238 \rightarrow .267$). Sharing the memory: fewer (.157).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.

One memory: the numbers

Memory studies $0+0$, $5+0$, $1+1$, then $100+100$, $100+200$.

	EPAM	ACT-R, separate table	ACT-R, one memory
$0+0$ answered 5 (retrieves $5+0$)	0	0	126/2167
$150+100$: solver invoked	1/46	$\approx .407$	$\approx .407$
solver correct, *given* invoked	$\approx .656$	$\approx .656$	$\approx .385$
correct overall	$\approx .014$	$\approx .267$	$\approx .157$
errors on practiced $100+100$	$\approx .0073$	$\approx .334$	$\approx .386$

- The middle column is a control: same five items in memory, but the solver doesn't use it.
- Extra items alone: *more* correct ($.238 \rightarrow .267$). Sharing the memory: fewer ($.157$).
- ACT-R without retrieval as use; finger counting is an implausible column fallback.

Here are the numbers, with the memory holding three digit pairs and the two problems. EPAM recognizes every pair as itself, so nothing changes from before. ACT-R confuses pairs: zero plus zero sometimes retrieves five plus zero and is answered five. Two things changed at once for ACT-R, the memory holds more items and the solver uses it, so the middle column is a control: the same five items in memory, but the solver keeps its separate table. The extra items alone actually raise correct answers a little, from about twenty-four to twenty-seven percent, because more problems reach an accurate solver. It's the solver's use of the memory that brings them down to about sixteen: the solver's accuracy when invoked drops from sixty-six percent to under forty. Two caveats: this is ACT-R without retrieval changing memory, and finger counting is an implausible fallback for someone doing three-digit addition.

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_q)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	$45/46$, $1/46$, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

For credibility, it's essential to say what each result rests on. Some are library facts, like the bridge and well-formedness: needed, but not psychology. Some are compositional deductions that hold at every setting, like solver independence. Some are conditional, like confident errors, and the conditions are where the science is: the proofs say exactly which premise a failed test would implicate. And some are just numbers at chosen settings. None of these is yet an empirical finding about people, and the numbers are illustrations, not fits to children's data.

What each result rests on

Kind	Examples
Library facts	the bridge; the composed trial is a probability distribution
Compositional deductions (every setting)	whole-problem errors independent of the solver; solver use is $(1 - R_s)^n$
Conditional	confident errors; two sources of error; any memory with three laws
At chosen settings	45/46, 1/46, $\approx .656$, $\approx .014$, the ACT-R column

None of these is yet an empirical finding about people. The numbers are illustrations, not fits.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Checking the formalization: a blind simulation

- A prose specification, no Lean; preregistered checks; a fresh agent wrote a simulator from the prose alone.
- **Agreement**: every check whose hypothesis arose passed; 243 frequencies matched Lean exactly.
- **Found what proof didn't**:
  - five theorems' hypotheses were *unrealized* in the tested learning histories (not shown unreachable);
  - a gap at zero total strength;
  - a predicted learning sequence depends on an unstated assumption: a new problem's initial associations.

Lean checks that the theorems follow from the definitions, not that the definitions say what the prose model says. So we checked that link independently. A prose specification with no Lean in it, checks preregistered before any simulator existed, and a fresh AI agent that wrote a simulator from the prose alone. Every check whose hypothesis arose passed, and two hundred forty-three outcome frequencies matched Lean's exact values. And it found things the proofs didn't. Five theorems had hypotheses that never arose in the learning histories we tested; that doesn't show they're unreachable, only that the theorems said little about those nets. There's a gap at zero total strength. And a predicted learning sequence turns out to depend on an assumption the model never stated: the initial associations of a newly discriminated problem.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Where does this sit? There's a lot of prior formal work on cognitive architectures: a specification of Soar in Z, the Sceptic specification language, formal semantics and model checking for ACT-R, and recently Soar models compiled for the PRISM probabilistic model checker. It helps to separate three approaches. Simulation samples executions. Probabilistic model checking analyzes a specified model exhaustively; that's real formal verification, not simulation, and it can treat parameters symbolically. Deductive proof establishes results for whole classes of models that satisfy theoretical assumptions. So the contrast isn't simulation versus verification; it's model verification versus theoretical deduction, and they're complementary. CogLean doesn't introduce formal methods to cognitive science. What's new is the combination: a shared, cumulative library with reusable proofs, composition across theories, and deductive discovery.

Relationship to cognitive architectures

- Not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present example composes locally, for one task, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

Relationship to cognitive architectures

- Not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present example composes locally, for one task, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

And to be clear about cognitive architectures. CogLean isn't proposed as an alternative to Soar, ACT-R, or the Common Model of Cognition. It's a possible formal foundation for expressing, composing and reasoning about their mechanisms and architectural commitments. The present example composes mechanisms locally, for one task, without committing to any general architecture. The longer-term aim is to formalize architectural constraints themselves, so that their implications can be proved and alternative architectures compared within one shared framework.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

What a proof shows, and what it doesn't

A word on what proofs do and don't show. Lean guarantees validity. It doesn't guarantee that the premises describe people, and it doesn't guarantee explanation, because the premises might already contain the phenomenon. We had exactly that case: a serial-position result whose mechanism was a bowed schedule, so proving a bowed curve was valid and nearly empty. It's now derived from an attention rule over anchors, which also gives the isolation effect, a second consequence of the same mechanism. The library keeps an explicit audit that sorts its results as definitional, close, derived or compositional. That's a judgment, written down in the open.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

What CogLean is, and isn't

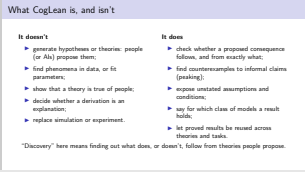
It doesn't

- generate hypotheses or theories: people (or AIs) propose them;
- find phenomena in data, or fit parameters;
- show that a theory is true of people;
- decide whether a derivation is an explanation;
- replace simulation or experiment.

It does

- check whether a proposed consequence follows, and from exactly what;
- find counterexamples to informal claims (peaking);
- expose unstated assumptions and conditions;
- say for which class of models a result holds;
- let proved results be reused across theories and tasks.

"Discovery" here means finding out what does, or doesn't, follow from theories people propose.



So, to be clear about what this is and isn't. CogLean doesn't generate hypotheses or theories; people, or AIs, propose them. It doesn't find phenomena in data or fit parameters. It doesn't show that a theory is true of people, and it can't decide whether a derivation is an explanation; that's a scientific judgment. And it doesn't replace simulation or experiment. What it does is check whether a proposed consequence actually follows, and from exactly which premises. It finds counterexamples to informal claims, like the peaking claim. It exposes assumptions and conditions that theories leave unstated. It says for which class of models a result holds. And it lets proved results be reused across theories and tasks. When I say discovery, that's what I mean: finding out what does, or doesn't, follow from theories that people propose.

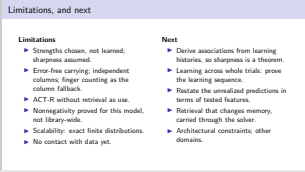
Limitations, and next

Limitations

- Strengths chosen, not learned; sharpness assumed.
- Error-free carrying; independent columns; finger counting as the column fallback.
- ACT-R without retrieval as use.
- Nonnegativity proved for this model, not library-wide.
- Scalability: exact finite distributions.
- No contact with data yet.

Next

- Derive associations from learning histories, so sharpness is a theorem.
- Learning across whole trials: prove the learning sequence.
- Restate the unrealized predictions in terms of tested features.
- Retrieval that changes memory, carried through the solver.
- Architectural constraints; other domains.



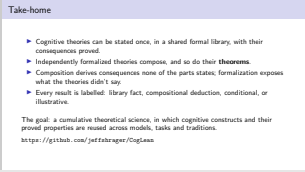
The limitations are real. The associative strengths are chosen, not learned, and sharpness, the key premise of confident errors, is assumed. Carrying is error-free and columns are independent, and the columns fall back on finger counting, which no one doing three-digit addition would do. ACT-R is represented by its retrieval rule only: retrieving something doesn't change its memory, so all the ACT-R numbers are for that version. Nonnegative weights are proved for this model but not yet across the library. Exact finite distributions raise scalability questions. And there's no contact with data yet. Next: derive the associations from learning histories, so that sharpness becomes a theorem instead of an assumption; add learning across whole trials, so the predicted learning sequence can be proved; restate the predictions whose hypotheses weren't realized; let retrieval change memory, carrying the memory's state through the solver; and move on to architectural constraints and other domains.

Take-home

- Cognitive theories can be stated once, in a shared formal library, with their consequences proved.
- Independently formalized theories compose, and so do their **theorems**.
- Composition derives consequences none of the parts states; formalization exposes what the theories didn't say.
- Every result is labelled: library fact, compositional deduction, conditional, or illustrative.

The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`



To sum up. Cognitive theories can be stated once, in a shared formal library, with their consequences proved rather than simulated. Independently formalized theories compose, and so do their theorems. Composition derives consequences that none of the parts states, and formalization exposes things the theories never said. And every result is labelled: a library fact, a compositional deduction, a conditional result, or an illustration. The goal is a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and research traditions, the way mathematical results are reused in Mathlib. One example doesn't establish that. It shows that the approach can produce results of the right kind.